OS Project 1 - Mallware Cop

A cybersteps project

Date: 26. October 2025

Participants: Gregor

References: https://github.com/ggrl/mallwarecop

Executive summary:

The goal is to build a process-monitoring tool in Python. It should watches the system-processes and also investigates the excutables behind the processes. It should check for suspicious behavior and take action against potentially malicious programs.

Step 1: Simple CLI Process Monitor

I first made a python script (process_scanner.py) which scans running processes using the psutil-module.

```
for p in psutil.process_iter(attrs=None):
```

I then added a function, which computes a suspicion-score based on different rules. (for example: high memory or cpu usage, system-like names but not in system-path, network-connectivity but not in whitelisted-path, contains specific keyword etc.)

At a specific score-threshold a process is flagged and the user can choose different actions to perform on these flagged processes (trust, kill, quarantine).

Trust will add the process name and the sha256-hash of the executable to a log-file. (whitlist.log)
This ensures that the application is actually the real application and it wasn't tampered with.
Kill will terminate the process. (With a safety-feature to prevent to terminate specific whitelisted system-processes)
Quarantine will move the executable to a specific folder.

Step 2: Add GUI

To make the process-monitor more interactive to give a better overview over the processes, I decided to add a graphical user interface using the tkinter-module.

I created a new main.py, which contains the GUI and borrowed functions from the process_scanner.py (like the trust, kill, quarantine-functions)

The GUI lists all running processes on the machine in an table with PID, Name, User, CPU%, Memory, executable-path and suspicion-score. You can also sort the processes by clicking the top of the columns.
It refreshes every 2 seconds. Every 60 seconds the application performs a heavy scan, which computes the suspicion-score for each running process.

Right-clicking a process opens a menu, which lets you kill, trust or quarantine the process. This calls the previously build functions of process_scanner.py.

You can also choose the option 'Info' which opens a popup-window with information about the selected process.

Step 3: Connect to VirusTotal-API

The next step was to add connectivity to the VirusTotal-API. First I planned using the vt-py-module for that. But I read that the implementation on Windows-ARM-Versions can be difficult and I also didn't needed it, because all I wanted to do was checking sha256-hashes of executables.
This could be faster done with a simple request-function:

```
def vt_scan(exe):

    sha256 = sha256_of_file(exe)
    print(exe)
    print(sha256)
    url = f"https://www.virustotal.com/api/v3/files/{sha256}"
    r = requests.get(url, headers=headers)
    if r.status_code == 200:
        data = r.json()
        stats = data["data"]["attributes"]["last_analysis_stats"]
        print(f"{os.path.basename(exe)}: {stats['malicious']} detections")
        dict1 = {"sha256": sha256, "malicious": stats['malicious'],
"suspicious": stats['suspicious'], "timestamp": int(time.time())}
        with open("vt_scan.json", "r") as log:
            json_data = json.load(log)
        json_data.append(dict1)
        with open("vt_scan.json", "w") as log:
            json.dump(json_data, log)
        return stats
    elif r.status_code == 404:
        print(f"{exe} not found on VT")
```

```
    else:
        print(f"Error {r.status_code}: {r.text}")
```

This solution is also better for cross-platform-usability.
Every VT-scan-result is also added in a file, to reduce the amount of scanning needed.
Since the free version of the virustotal-api only allows for limited scans, I added 4 automatic scans per heavy scan. Additionally applications only get scanned with a suspicion-score > 0 and if not whitelisted (trusted).
You can also make a manual scan via the right-click-menu.

Step 4: Automated reactions

The next step was to add automated actions based on policy. I added popup-warnings for a high suspicion-score, high ram-usage (over 500MB) and virus-total-findings.
Every warning is also logged in a log-file.
I left the decision to suspend or quarantine a process at the user, but it would be easy to call the terminate or quarantine-function automatically.

All files:

- main.py - contains GUI and base logic
- process_scanner.py - former cli-version, contains functions for the main.py (It is not possible to run it independetly anymore)
- whitelist.log - contains names and exe-hashes for manually trusted applications
- warning.log - contains warning-logs
- vt_scan.json - contains exe-hashes and results of virustotal-scans

Possible improvements for the future

More automation

As mentioned above, I could automatically terminate processes.

Beautification

The warning-popups could be prettier or the warnings could be better integrated in the GUI as a banner for example.

Program hardening

From time to time the program can get unresponsive, while performing the heavy scan. I wasn't able to exactly determine the cause, but I suspect the scan-thread gets stuck somehow.